

Research Copilot — Interview Prep

Questions Only — RAG, Agentic Systems & Production LLM Engineering

50 scenario-based questions across 12 topics, grounded in real architecture decisions, real bugs, and real eval results from building the Research Copilot project (Weeks 1). Answers are in the companion PDF — attempt these first.

Topics to Learn

Study these before attempting the questions. Each maps directly to a section below and to real decisions/bugs from this project.

Vector Databases & ANN Search	HNSW, approximate vs. exact nearest-neighbor, quantization, sharding, Qdrant vs. pgvector vs. Weaviate vs. Pinecone trade-offs.
Embedding Models	Bi-encoders vs. cross-encoders, asymmetric instruction prefixes, domain mismatch, dimensionality trade-offs, MTEB benchmarking.
Hybrid Search & Fusion	BM25/sparse retrieval, dense retrieval, Reciprocal Rank Fusion (RRF), why ranks (not raw scores) are fused.
Chunking Strategies	Recursive/structure-aware chunking, chunk-size tuning, semantic chunking, Parent-Child chunking, Contextual Retrieval, Late Chunking.
Reranking	Cross-encoder rerankers, why they outperform bi-encoders at small scale, LLM-as-reranker, domain mismatch in rerankers.
Agentic Loops & Tool Calling	Hand-rolled vs. framework-based (LangGraph) agent loops, native function calling, iteration caps, conversation state, malformed tool calls.
Citation, Faithfulness & Hallucination Mitigation	Structured-output citation schemas, deterministic verification, post-hoc NLI/embedding attribution, RAGAS faithfulness vs. context precision/recall, abstention design.
RAG Evaluation Design	Golden sets, hand-labeling vs. synthetic labels, precision@k/recall@k/MRR, ablation studies, adversarial/partial-coverage test design.
LLM API Reliability Engineering	Rate limits vs. daily quotas, retry/backoff strategy design, timeout handling, silent model-alias regressions, defensive error handling.

Cost & Latency Trade-offs	Per-call latency compounding in multi-call agent loops, model selection (thinking vs. lite models), cost-at-scale estimation.
Financial Document Domain Knowledge	Inline XBRL, SEC filing structure, restatement/version drift, why dense financial filings stress-test RAG systems differently than prose.
System Design & Architecture Decision-Making	ADR-driven decision documentation, incremental build philosophy, scaling bottleneck analysis, cross-domain architecture reuse.

Contents

A. Vector Databases & Retrieval Architecture

A1, A2, A3, A4, A5, A6

B. Embedding Models

B1, B2, B3, B4, B5

C. Hybrid Search & Fusion

C1, C2, C3

D. Chunking Strategies

D1, D2, D3, D4, D5, D6

E. Reranking

E1, E2, E3, E4

F. Agentic Loops & Tool Calling

F1, F2, F3, F4, F5

G. Citation, Faithfulness & Hallucination Mitigation

G1, G2, G3, G4, G5

H. RAG Evaluation Design

H1, H2, H3, H4, H5, H6

I. LLM API Reliability Engineering

I1, I2, I3, I4, I5

J. Cost & Latency Trade-offs

J1, J2, J3

K. Financial Document Domain Knowledge

K1, K2, K3

L. Capstone System Design Scenarios

L1, L2, L3, L4

A. Vector Databases & Retrieval Architecture

A1. You're building a RAG system that needs to scale from 100K to 50M chunks over 18 months, on a team of 2 engineers with no dedicated DevOps, and a \$500/month infra budget. You initially chose Qdrant self-hosted via Docker on a single VM. At what point would you need to change your architecture, and what would you change first: the vector store, the hosting model, or something else? Justify with specific technical triggers, not just "when it gets slow."

A2. Contrast Qdrant, pgvector, Weaviate, and Pinecone across: operational ownership, hybrid search support, and learning value for an engineer trying to deeply understand retrieval internals. In what specific scenario would pgvector actually outperform a dedicated vector DB, not just "be simpler"?

A3. Your Qdrant collection uses HNSW indexing. Explain, in your own words, why HNSW is approximate and not exact nearest-neighbor search, and describe one concrete scenario in this project (SEC filings retrieval) where that approximation could cause a wrong answer to surface without any bug in your code.

A4. You've been asked to add scalar quantization to reduce Qdrant's memory footprint by 4x. What accuracy/recall tradeoff should you expect, and how would you validate whether that tradeoff is acceptable for THIS project specifically, using infrastructure you already built?

A5. Your vector store is showing p99 query latency of 800ms at 2M vectors, well above your 200ms SLA. Walk through your diagnostic process: what's the first thing you check, second, third? Where does sharding fit into that list, and where does it NOT belong?

A6. Why did the project keep vectors/metadata in Qdrant, separate from a relational database, rather than using pgvector to have "one source of truth" in Postgres? What operational cost does that separation introduce that isn't obvious until you've actually built ingestion pipelines for both structures?

B. Embedding Models

B1. Explain why BGE-family embedding models require an instruction prefix ("Represent this sentence for searching relevant passages:") for QUERIES but not for documents. What class of embedding models does NOT have this asymmetry, and what's the architectural reason?

B2. You swap your embedding model from bge-base-en-v1.5 (768-dim) to a new model with 1536 dimensions that scores higher on MTEB. List every downstream component in a hybrid-search RAG pipeline (like this project's) that would break or need re-work, beyond "just re-embed everything."

B3. Gemini's free-tier embedding API was wired in as a configurable alternative to a local BGE model. If asked "why not just use the API embedding model as the default since it's higher quality," what's your answer, and what assumption in that question is wrong?

B4. What's the actual difference between a bi-encoder and a cross-encoder at the architecture level, not just "one is faster"? Why can't you use a cross-encoder for the initial retrieval step over millions of documents, even if you had unlimited compute?

B5. Your embedding model was trained predominantly on web text and general documents. You're now embedding SEC 10-K filings full of financial jargon, ticker symbols, and regulatory boilerplate. What specific failure modes would you expect from a domain mismatch, and what are two ways to address it that DON'T involve fine-tuning your own embedding model?

C. Hybrid Search & Fusion

C1. Explain Reciprocal Rank Fusion (RRF) mathematically, not just "it combines rankings," and explain why RRF uses RANKS rather than raw similarity SCORES to combine dense and sparse results. What problem would arise if you tried to combine raw cosine similarity and raw BM25 scores directly by weighted sum?

C2. In this project's retrieval ablation, Hit@5 went dense-only 14.3% → +hybrid 28.6% → +reranking 32.1% (at the original 500-token chunk size). Explain, mechanistically, why BM25/sparse search would catch cases that dense embedding search misses, using a concrete example from a financial-filing corpus.

C3. If your sparse (BM25) component and your dense component disagree strongly and consistently on the same class of queries, what would that tell you about your corpus or your queries, and how would you diagnose which one is "more right" without just picking a favorite?

D. Chunking Strategies

D1. This project found that reducing chunk size from 500 tokens to 200 tokens raised Hit@5 from 32.1% to 53.6%. Explain the actual mechanism: why does a larger chunk size hurt embedding-based retrieval, even when the correct answer is fully contained within the larger chunk?

D2. You just cut chunk size from 500 to 200 tokens to fix a retrieval-precision problem, and it worked. Six weeks later, your citation-synthesis quality has degraded — the model is generating oddly fragmented, incomplete-sounding answers. What's the likely mechanism connecting your chunking change to this new problem, and what's the standard architectural fix?

D3. Compare Parent-Child chunking and Contextual Retrieval as two different fixes for the SAME underlying problem (facts losing meaning when isolated into small chunks). What is that shared underlying problem, and why are these two fixes NOT redundant with each other?

D4. Why was Late Chunking (Jina AI's technique) rejected for this project specifically, and what would have to change about the project's existing architecture for Late Chunking to become viable?

D5. You're chunking a corpus of technical API documentation (short, code-heavy pages with lots of headers) versus a corpus of financial 10-K filings (long, dense paragraphs mixed with huge financial tables). Would you use the same `chunk_size` and `overlap` parameters for both? Justify chunk size specifically in terms of what kind of "fact isolation" failure each corpus is prone to.

D6. In this project, the same "Total deposits of \$X billion, up \$Y billion from December 31, 2022" sentence structure appeared in MULTIPLE unrelated contexts across a single 10-K (total deposits, total assets, total loans, stockholders' equity, all using nearly identical phrasing). Why does this specific pattern hurt BOTH sparse (BM25) and dense retrieval simultaneously, rather than being a problem for only one of them?

E. Reranking

E1. Explain, at an architectural level, why a cross-encoder reranker can afford to be much more accurate than the bi-encoder used for initial retrieval, given that both are "just" transformer models scoring relevance.

E2. At 500-token chunks, adding a reranker improved Hit@5 from 28.6% to 32.1% — a clear win. At 200-token chunks, reranking improved Hit@5 (46.4% → 53.6%) but WORSENE MRR (0.354 → 0.320) and Hit@1 (28.6% → 21.4%) compared to hybrid search alone. Propose two different hypotheses for why a reranker's benefit would change character when chunk size changes, and describe an experiment to distinguish between them.

E3. Why was LLM-as-reranker explicitly excluded from the production runtime path in this project, even though it was evaluated as an ablation? What's the actual bottleneck, and how does that bottleneck change for a low-QPS internal tool instead of a consumer product?

E4. If your reranker is trained on general web-relevance data and you deploy it on a financial-filings domain, what specific failure mode would you expect at inference time, and how would you detect it using only the metrics your eval harness already computes, without adding new instrumentation?

F. Agentic Loops & Tool Calling

F1. Explain the difference between "automatic function calling" and a hand-rolled agent loop. What do you lose in terms of learning/control by using automatic function calling, and what do you gain in terms of engineering time?

F2. Your hand-rolled agent loop has `MAX_SEARCH_ITERATIONS=4` as a hard cap. A user asks a genuinely complex multi-hop question that would benefit from 6 search iterations to fully answer. What are three different design responses to this situation, and what are the tradeoffs of each?

F3. Why does this project's agent loop maintain conversation HISTORY across search iterations rather than treating each search call as a stateless, independent request? What would break if you made it stateless?

F4. At what point does a single-tool agent loop justify migrating to a graph-based framework like LangGraph? Name three concrete signals in your own codebase that would tell you "now is the time," not just "frameworks are generally good."

F5. During testing, the model called the search tool without providing the required query argument, causing an unhandled crash. Beyond the specific defensive fix, what does this incident tell you about a broader principle for building production systems on top of LLM tool-calling, regardless of provider?

G. Citation, Faithfulness & Hallucination Mitigation

G1. Explain the difference between: (a) prompting the model to self-report inline citations, (b) structured output with a per-claim citation schema plus deterministic verification, (c) post-hoc NLI/embedding-based attribution after free-form generation. For each, name the specific failure mode it does NOT protect against.

G2. Your citation-verification eval found that for two adversarial questions, the system produced a confident, cited-looking answer instead of correctly abstaining. Both questions concerned data that existed ONLY as an image in the source document. Walk through the most likely failure point: retrieval, synthesis, or verification? How would you instrument the pipeline to find out definitively?

G3. Why does this project's `_verify()` function check citations against the actually-retrieved evidence set rather than trusting the LLM's self-reported claim about which chunk supports which sentence? Give a concrete example of a citation that would pass this deterministic check but still be substantively wrong.

G4. What is RAGAS's "faithfulness" metric actually measuring, in contrast to "context precision" and "context recall"? Give an example answer that would score well on faithfulness but poorly on context recall, and vice versa.

G5. A stakeholder asks: "Why does the system sometimes say 'I don't have enough information' when the answer is clearly somewhere in the corpus?" Using this project's actual architecture, list every layer where that failure could originate, and for each, name one metric from your eval harness that would help localize the fault.

H. RAG Evaluation Design

H1. Why is "hand-labeling ground truth against real documents" considered more valuable, from an interview-signal perspective, than generating a synthetic golden set entirely with an LLM? What specific failure mode of LLM-generated golden sets does hand-labeling avoid?

H2. Explain precision@k, recall@k, and MRR as used in this project's retrieval ablation. Construct a hypothetical scenario where Hit@5 is high but MRR is low, and explain what that tells you that Hit@5 alone would hide.

H3. Your retrieval ablation shows each layer helping (14.3% → 28.6% → 32.1% Hit@5). Your project lead says "great, ship it, 32% is fine since it's better than nothing." What is wrong with stopping the investigation there, and what did this project actually do instead when facing an unimpressive absolute number?

H4. Why does this project deliberately include "partial coverage" test questions in the eval set, rather than only including questions with fully verified, complete answers? What specific system behavior is this designed to catch?

H5. What is the difference between an ablation study and a golden-set eval, and why does this project need BOTH? Specifically, explain why the retrieval ablation could run entirely offline while the citation- verification pass could not.

H6. You have a fixed budget of \$200 in eval-time LLM API calls before a launch deadline. You have 200 golden-set questions but can only afford to run about 60 through the full pipeline. How do you decide which 60 to prioritize, and what's the risk of choosing badly?

I. LLM API Reliability Engineering

I1. Explain the practical difference between a "per-minute rate limit" and a "per-day quota" on a free-tier LLM API, and why the SAME retry-with- exponential-backoff strategy is appropriate for one and actively harmful for the other.

I2. A model alias like "gemini-flash-latest" silently began resolving to a slower, more expensive "thinking" model variant, with no error, warning, or code change on your end. Design a lightweight monitoring approach using only what this project already has, no new infrastructure, that would

have caught this automatically.

I3. Why is it dangerous to add a broad except Exception: continue around a loop processing untrusted LLM outputs, even though it "fixes" the immediate crash? What did this project do differently to avoid silently masking real bugs while still gaining resilience?

I4. You observe that ~43% of your eval questions are failing with DeadlineExceeded errors. Before writing a single line of retry logic, what are three hypotheses you should rule out first, and how would you test each one cheaply?

I5. This project's retry.py treats ResourceExhausted (rate limit) and DeadlineExceeded (gateway timeout) as retryable, but DailyQuotaExhausted as immediately fatal. What general principle determines whether an error should be retried, and how do you apply that principle to a new, previously-unseen error type you haven't hard-coded a rule for?

J. Cost & Latency Trade-offs

J1. In an agentic loop that may call an LLM 2-6 times per user question, why does per-call latency matter disproportionately more than in a single-call RAG system? Do the math: at 12 seconds/call versus 1 second/call, what's the user-facing latency difference for a question requiring 4 tool-call round-trips plus 1 synthesis call?

J2. You need to choose between a fast/cheap model for an agent's tool-calling loop and a slower/higher-quality model for final answer synthesis. Justify using two DIFFERENT models for these two roles within the same pipeline, and identify one risk this two-model approach introduces.

J3. What would "cost per 1,000 user questions" look like for this project's architecture, and what's the first thing that breaks if usage grew 100x? Is it the Gemini quota, the local compute, or something else?

K. Financial Document Domain Knowledge

K1. Explain what inline XBRL is and why SEC filings (post-2019 mandate) contain a large block of machine-readable tagging data interleaved with human-readable text. What specifically goes wrong if a naive HTML-to-text scraper doesn't account for this?

K2. A user asks your system "What was Company X's total deposits?" without specifying a date or filing period. Given that SEC filings routinely restate the SAME historical figure differently across different filings, what does a well-designed RAG system need to do differently here compared to answering a question about a static fact?

K3. Why is "regional banks in 2023" specifically a harder, more differentiated corpus for a RAG eval than "clean, well-structured single-topic documents," in terms of the actual retrieval/citation challenges it creates?

L. Capstone System Design Scenarios

L1. You're asked to redesign this Research Copilot to handle 10,000 documents (500x the current corpus) and 1,000 concurrent users, while keeping the \$0/month local-inference cost model. Walk through which THREE components would break first, in order, and what you'd change for each.

L2. A stakeholder asks you to add support for legal contracts to the SAME system, reusing as much infrastructure as possible. Which existing ADR decisions would need to be REVISITED for this new domain, and which would likely transfer unchanged?

L3. Your citation-verification eval revealed a 0% abstention rate on 2 tested adversarial questions. Your manager wants this fixed before the two remaining weeks end, alongside three other roughly-equal-priority items (finishing the eval set, adding the UI, building the semantic cache). Argue for where this fix should rank in priority.

L4. Design an experiment (not just "run RAGAS") to determine whether your reranker is HELPING or HURTING at your current chunk size, given the ambiguous MRR/Hit@1 vs Hit@5 result from this project. What additional data would you need that the current ablation doesn't already give you?